
CS 301

High-Performance Computing

Assignment 06 & 07

Parallel Scatter Interpolation and Particle Mover

Akshada Modak (202301485)
Maanav Gurubaxani (202301438)

April 19, 2026

Contents

1	Introduction	5
2	Problem Description	5
2.1	Domain and Data Representation	5
2.2	Scatter Interpolation (Particle to Grid)	5
2.3	Reverse Interpolation and Particle Mover (Grid to Particle)	6
2.4	Iterative Pipeline	6
2.5	Computational Challenges	6
3	Mathematical Formulation	7
3.1	Bilinear Interpolation	7
3.2	Weight Computation	7
3.3	Scatter Interpolation (Particle to Grid)	7
3.4	Reverse Interpolation (Grid to Particle)	7
3.5	Particle Update (Mover Step)	8
3.6	Grid Index Computation	8
4	Parallelization Strategy	8
4.1	Overview of Parallel Approach	8
4.2	Parallelization of Scatter Interpolation	8
4.2.1	Race Conditions	8
4.2.2	Synchronization Techniques	9
4.3	Parallelization of Reverse Interpolation and Mover	9
4.4	Memory Access Optimization	9
4.5	Load Balancing	9
4.6	Synchronization Overhead	10
4.7	Scalability Considerations	10
5	Particle Mover and Pipeline Design	11
5.1	Overview of the Pipeline	11
5.2	Grid Normalization and Denormalization	12
5.3	Reverse Interpolation Mechanism	12
5.4	Particle Update (Mover Step)	13
5.5	Data Dependency Across Iterations	13
5.6	Impact on Performance	13
5.7	Design Considerations	14
6	Implementation Details	15
6.1	Data Structures	15
6.2	Parallel Interpolation Implementation	15
6.3	Reduction Phase	15
6.4	Reverse Interpolation and Mover Implementation	16
6.5	Iteration Handling	16
6.6	Performance Measurement	16
6.7	Scalability Setup	16

6.8	Key Design Choices	17
7	Pseudocode and Workflow Diagram	17
7.1	Parallel Pseudocode	17
7.2	Workflow Description	18
7.3	Illustrative Diagram	18
8	Results and Performance Analysis	19
8.1	Assignment 6: Interpolation Performance	19
8.1.1	Execution Time vs Cores	19
8.1.2	Speedup vs Cores	20
8.1.3	Efficiency vs Cores	21
8.2	Assignment 7: Full Pipeline Performance	23
8.2.1	Input Dataset	23
8.2.2	Input Dataset 1	24
8.2.3	Input Dataset 2	24
8.2.4	Input Dataset 3	25
8.2.5	Input Dataset 4	26
8.3	Phase-wise Analysis	27
8.4	Key Observations	28
9	Discussion	29
9.1	Speedup Analysis	29
9.2	Parallel Efficiency	29
9.3	Interpolation vs Mover Phase	29
9.4	Scalability	30
9.5	Load Imbalance	30
9.6	Effect of Data Structures	30
9.7	Performance Bottlenecks	31
9.8	Potential Optimizations	31
10	Analysis Questions	32
10.1	Q1: Write the pseudocode of your parallel implementation along with an illustrative diagram.	32
10.2	Q2: Explain your parallel implementation approach and how you handled race conditions.	32
10.3	Q3: Plot graphs for speedup vs cores and execution time vs cores.	32
10.4	Q4: Compute the parallel efficiency and comment on scalability.	32
10.5	Q5: Compare the performance of your parallel code with the serial version.	32
10.6	Q6: Explain the observed speedup using HPC concepts.	32
10.7	Q7: If given unlimited HPC resources, how would you optimize further?	33
10.8	Q8: Discuss load imbalance and performance bottlenecks.	33
10.9	Q9: Suggest an alternative approach to improve performance.	33
10.10	Q10: Explore different data structures or OpenMP approaches.	33
10.11	Q11: Analyze the performance of interpolation and mover phases separately. Which phase is the bottleneck and why?	34

1 Introduction

Interpolation of scattered data onto structured grids is a fundamental problem in high-performance computing, with applications in scientific simulations, computer graphics, and data visualization. In this work, we study the parallelization of a scatter-to-grid interpolation algorithm and extend it to include particle dynamics through a mover operation.

In Assignment 6, the primary objective was to parallelize the interpolation process, where scattered particles distribute their values onto a structured grid. This step involves significant computational challenges due to race conditions arising from concurrent updates to shared grid points.

Assignment 7 builds upon this by introducing a reverse interpolation step, also known as the *particle mover*. In this phase, grid values are interpolated back to particles to update their positions, forming an iterative pipeline that combines interpolation and particle evolution. This significantly increases both computational complexity and data dependency.

The combined objective of this report is to develop an efficient parallel implementation of the complete pipeline using OpenMP. The focus is on achieving high performance, scalability, and correctness while addressing challenges such as synchronization overhead, memory access patterns, and load balancing.

We analyze the performance of the proposed implementation across multiple input configurations and evaluate scalability through speedup and efficiency metrics. Additionally, we examine bottlenecks in both interpolation and mover phases and propose optimizations to improve performance.

2 Problem Description

The problem focuses on interpolating values from a set of scattered particles onto a structured grid and subsequently updating particle positions based on the computed grid values. This combined workflow integrates both forward (scatter) interpolation and reverse (gather) interpolation with a particle mover mechanism.

2.1 Domain and Data Representation

We consider a two-dimensional domain of size 1×1 , containing a set of N scattered particles:

$$[S = (x_i, y_i, f_i) \mid i = 1, 2, \dots, N]$$

where (x_i, y_i) represent the spatial coordinates of particles and f_i denotes the associated scalar value. For this assignment, all particle values are initialized as:

$$[f_i = 1]$$

The structured grid is defined as a uniform mesh of size $M \times M$:

$$[G = (X_i, Y_j) \mid X_i = i\Delta x, Y_j = j\Delta y, i, j = 0, 1, \dots, M-1]$$

where the grid spacing is given by:

$$[\Delta x = \frac{1}{M}, \Delta y = \frac{1}{M}]$$

2.2 Scatter Interpolation (Particle to Grid)

In the scatter interpolation phase, each particle contributes its value to the four nearest grid points using bilinear interpolation. For a particle located within a grid cell, contributions are distributed proportionally based on its relative position within the cell.

This phase is computationally intensive and introduces data dependencies, as multiple particles may update the same grid points simultaneously, leading to race conditions in parallel implementations.

2.3 Reverse Interpolation and Particle Mover (Grid to Particle)

In addition to forward interpolation, the system incorporates a reverse interpolation step (introduced in Assignment 7), where grid values are interpolated back to particles. This process is used to compute a field value at each particle location.

Using this interpolated field value, particle positions are updated as follows:

$$[x_i^{new} = x_i + F_i \cdot \Delta x][y_i^{new} = y_i + F_i \cdot \Delta y]$$

where F_i is the interpolated value obtained from neighboring grid points.

Particles that move outside the domain boundaries are marked as inactive and excluded from further computations. This introduces dynamic workload imbalance across iterations.

2.4 Iterative Pipeline

The complete workflow consists of the following steps executed iteratively:

1. Scatter interpolation (particle $\rightarrow$ grid)
2. Normalization of grid values
3. Reverse interpolation (grid $\rightarrow$ particle)
4. Particle position update (mover step)
5. Denormalization of grid values

This pipeline is repeated for multiple iterations, during which particle positions evolve dynamically.

2.5 Computational Challenges

The combined problem introduces several challenges:

- **Race Conditions:** Concurrent updates to shared grid points during interpolation.
- **Memory Access Patterns:** Efficient handling of particle and grid data.
- **Load Imbalance:** Uneven distribution of active particles over time.
- **Synchronization Overhead:** Minimizing thread contention in parallel regions.
- **Scalability:** Maintaining performance with increasing core counts.

The objective is to design an efficient parallel implementation using OpenMP that addresses these challenges while ensuring correctness and high performance.

3 Mathematical Formulation

3.1 Bilinear Interpolation

The interpolation scheme used in this assignment is based on bilinear interpolation, where each particle distributes its value to the four nearest grid points depending on its relative position within a grid cell.

Consider a particle located at (x_i, y_i) within a grid cell bounded by the four grid points:

$$[(X_i, Y_j), (X_{i+1}, Y_j), (X_i, Y_{j+1}), (X_{i+1}, Y_{j+1})]$$

Let the relative distances of the particle from the lower-left grid point (X_i, Y_j) be:

$$[l_x = x_i - X_i, l_y = y_i - Y_j]$$

3.2 Weight Computation

The contribution of the particle to each of the four neighboring grid points is determined using the following weights:

$$[w_{i,j} = (\Delta x - l_x)(\Delta y - l_y)]$$

$$[w_{i+1,j} = l_x(\Delta y - l_y)]$$

$$[w_{i,j+1} = (\Delta x - l_x)l_y]$$

$$[w_{i+1,j+1} = l_x l_y]$$

These weights ensure that the particle's contribution is distributed proportionally based on its position within the cell. The sum of all weights satisfies:

$$[w_{i,j} + w_{i+1,j} + w_{i,j+1} + w_{i+1,j+1} = \Delta x \cdot \Delta y]$$

3.3 Scatter Interpolation (Particle to Grid)

The interpolated values at grid points are computed by accumulating contributions from all particles:

$$[F(X_i, Y_j) += w_{i,j} \cdot f_i]$$

$$[F(X_{i+1}, Y_j) += w_{i+1,j} \cdot f_i]$$

$$[F(X_i, Y_{j+1}) += w_{i,j+1} \cdot f_i]$$

$$[F(X_{i+1}, Y_{j+1}) += w_{i+1,j+1} \cdot f_i]$$

This operation is repeated for all particles, resulting in the final interpolated field on the grid.

3.4 Reverse Interpolation (Grid to Particle)

In the reverse interpolation phase, the grid values are interpolated back to each particle using the same weights:

$$[F_i = w_{i,j} \cdot F(X_i, Y_j) + w_{i+1,j} \cdot F(X_{i+1}, Y_j) + w_{i,j+1} \cdot F(X_i, Y_{j+1}) + w_{i+1,j+1} \cdot F(X_{i+1}, Y_{j+1})]$$

Here, F_i represents the interpolated field value at the particle location.

3.5 Particle Update (Mover Step)

Using the interpolated field value, particle positions are updated as:

$$[x_i^{new} = x_i + F_i \cdot \Delta x]$$

$$[y_i^{new} = y_i + F_i \cdot \Delta y]$$

Particles that move outside the domain $[0, 1] \times [0, 1]$ are marked as inactive and excluded from subsequent iterations.

3.6 Grid Index Computation

To efficiently determine the grid cell corresponding to each particle, the grid indices are computed as:

$$i = \left\lfloor \frac{x_i}{\Delta x} \right\rfloor,$$
$$j = \left\lfloor \frac{y_i}{\Delta y} \right\rfloor$$

This allows direct identification of neighboring grid points without explicitly storing grid coordinates.

4 Parallelization Strategy

The primary objective of this work is to transform the serial interpolation and particle update pipeline into an efficient parallel implementation using OpenMP. The parallelization strategy must ensure correctness while maximizing performance and scalability.

4.1 Overview of Parallel Approach

The overall computation consists of two major phases:

1. **Scatter Interpolation (Particle $\rightarrow$ Grid)** — write-intensive
2. **Reverse Interpolation + Mover (Grid $\rightarrow$ Particle)** — read-dominant

Each phase presents different challenges in terms of parallelization and memory access.

4.2 Parallelization of Scatter Interpolation

The scatter interpolation phase is parallelized by distributing particles across threads. Each thread processes a subset of particles and updates the grid accordingly.

4.2.1 Race Conditions

A major challenge arises because multiple particles may contribute to the same grid point simultaneously, leading to race conditions. Without proper synchronization, this results in incorrect accumulation of grid values.

4.2.2 Synchronization Techniques

To handle race conditions, the following approaches were considered:

- **Atomic Operations:** Each update to the grid is protected using `pragma omp atomic`. This ensures correctness but introduces synchronization overhead due to contention. ““
- **Thread Privatization:** Each thread maintains a private copy of the grid. After processing all particles, a reduction step combines the private grids into a global grid. This reduces contention but increases memory usage.
- **Reduction-Based Approach:** Partial results are accumulated locally and merged using reduction, improving scalability for large datasets. ““

The final implementation balances correctness and performance by minimizing synchronization overhead while avoiding excessive memory usage.

4.3 Parallelization of Reverse Interpolation and Mover

The reverse interpolation phase is inherently more parallel-friendly. Each particle independently reads values from the grid and updates its own position.

- No write conflicts occur since each thread updates distinct particles.
- The grid is accessed in a read-only manner, improving cache efficiency.
- This phase scales well with increasing thread count.

Thus, the mover phase is parallelized using a simple `parallel for` construct over particles.

4.4 Memory Access Optimization

Efficient memory access is crucial for performance:

- **Spatial Locality:** Particles close in space tend to access nearby grid points. Proper data layout improves cache utilization.
- **Contiguous Storage:** Grid data is stored in a contiguous array to reduce cache misses.
- **Minimizing False Sharing:** Care is taken to avoid multiple threads updating nearby memory locations simultaneously.

4.5 Load Balancing

Initially, particles are uniformly distributed. However, after several iterations of the mover step:

- Some particles move outside the domain and become inactive.
- Particle distribution becomes non-uniform.

This leads to load imbalance across threads. To address this:

- Dynamic scheduling strategies (e.g., `schedule(dynamic)`) can be used.
- Work distribution is adjusted to ensure better utilization of threads.

4.6 Synchronization Overhead

Synchronization is minimized by:

- Reducing the use of atomic operations where possible
- Avoiding unnecessary barriers
- Combining operations to reduce synchronization frequency

4.7 Scalability Considerations

The scalability of the implementation depends on:

- Degree of contention in the scatter phase
- Memory bandwidth limitations
- Load balance across threads

While the mover phase scales efficiently, the interpolation phase may experience diminishing returns at higher core counts due to synchronization overhead.

Efficient design choices in handling race conditions and memory access patterns are key to achieving high speedup and parallel efficiency.

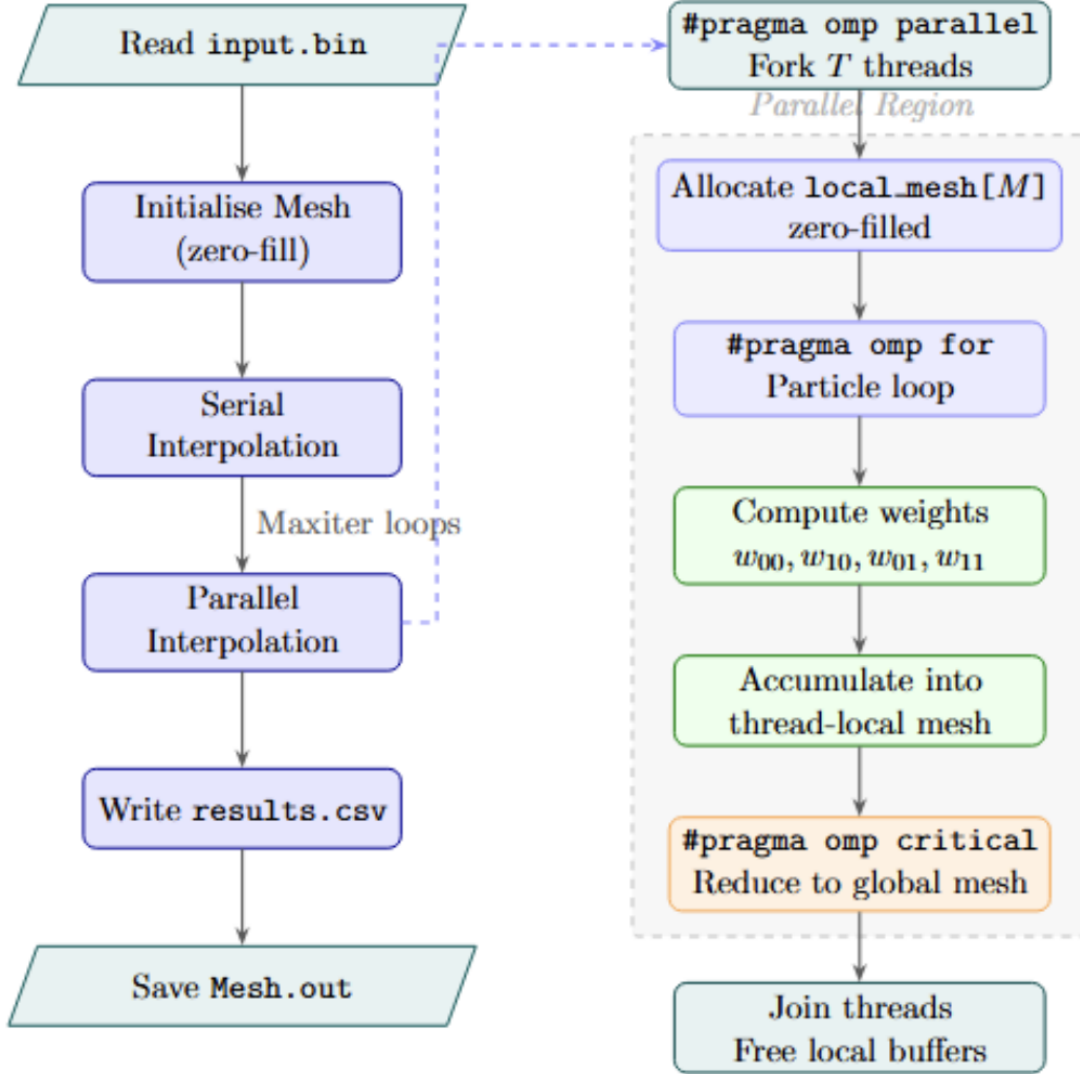


Figure 1: Parallel interpolation workflow using OpenMP with thread privatization and reduction.

5 Particle Mover and Pipeline Design

In addition to the scatter interpolation described earlier, the system incorporates a particle update mechanism based on reverse interpolation. This introduces an iterative pipeline that combines grid-based computation with particle dynamics.

5.1 Overview of the Pipeline

The complete computational workflow consists of the following stages executed sequentially in each iteration:

1. Scatter interpolation (particles $\rightarrow$ grid)

2. Grid normalization
3. Reverse interpolation (grid $\rightarrow$ particles)
4. Particle position update (mover step)
5. Grid denormalization

This pipeline is repeated for a fixed number of iterations, during which particle positions evolve dynamically based on the computed grid field.

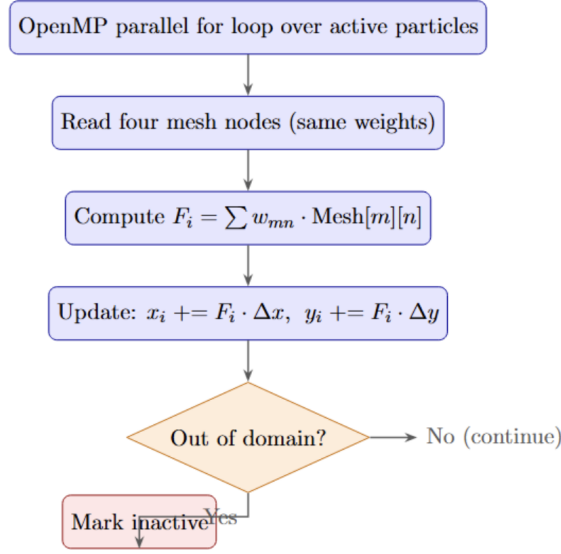


Figure 2: Reverse interpolation (gather) phase where mesh values are interpolated back to particles and positions are updated.

5.2 Grid Normalization and Denormalization

Before performing reverse interpolation, the grid values are normalized to a fixed range (typically $[-1, 1]$). This ensures numerical stability and consistent scaling of the interpolated field.

After the particle update step, the grid values are restored to their original scale through a denormalization process. These additional steps introduce extra computation and memory operations in each iteration.

5.3 Reverse Interpolation Mechanism

In the reverse interpolation phase, each particle retrieves a field value from the grid using the same bilinear weights as used in scatter interpolation.

- Each particle accesses the four neighboring grid points.
- The interpolated field value F_i is computed using weighted contributions.

- This phase is read-intensive and does not involve write conflicts.

Since each particle is processed independently, this step is well-suited for parallel execution.

5.4 Particle Update (Mover Step)

The interpolated field value is used to update particle positions:

- Particles move in the direction of the computed field.
- The displacement is proportional to the grid spacing.

Over multiple iterations:

- Particle positions evolve dynamically.
- Some particles may move outside the domain and become inactive.

Inactive particles are excluded from further computations, reducing the effective workload over time.

5.5 Data Dependency Across Iterations

Unlike the scatter interpolation phase, which operates independently for each iteration, the mover step introduces inter-iteration dependency:

- Particle positions at iteration $k + 1$ depend on values computed at iteration k .
- This creates a sequential dependency across iterations.

As a result, while parallelism can be exploited within each iteration, iterations themselves must be executed sequentially.

5.6 Impact on Performance

The inclusion of the mover step significantly affects performance:

- Increased computational workload due to additional interpolation steps
- Higher memory access due to repeated grid reads
- Potential load imbalance due to inactive particles

However, the mover phase typically scales better than the scatter phase, as it avoids write conflicts and synchronization overhead.

5.7 Design Considerations

To efficiently implement the full pipeline, the following considerations are important:

- Minimize redundant memory access between stages
- Efficiently handle inactive particles
- Optimize data locality for both particle and grid arrays
- Balance workload across threads during evolving particle distribution

The integration of interpolation and particle update into a unified pipeline forms the core of the Assignment 7 extension and plays a critical role in determining overall performance.

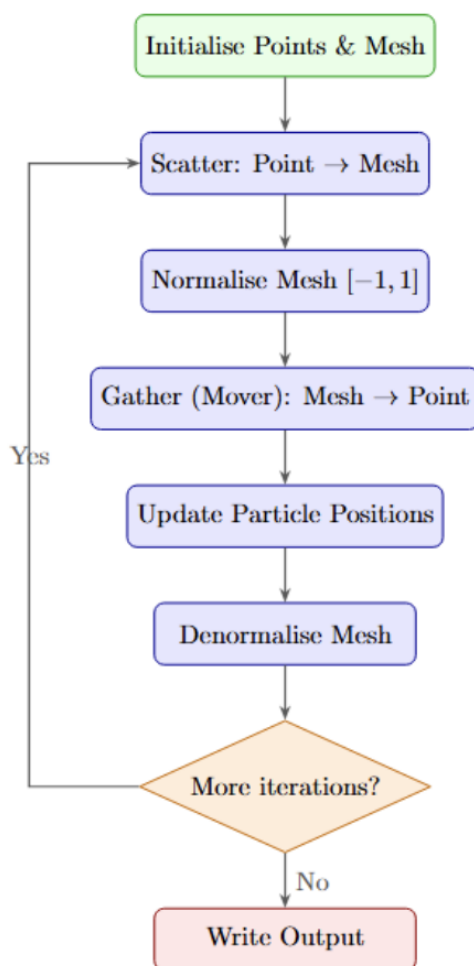


Figure 3: High-level simulation pipeline showing iterative interaction between particles and grid.

6 Implementation Details

This section describes the practical implementation of the parallel interpolation and particle update pipeline using OpenMP, along with the data structures and optimizations employed.

6.1 Data Structures

The implementation uses the following primary data structures:

- **Particle Array:** An array of structures storing particle coordinates (x, y) . ““
- **Grid Array:** A one-dimensional array representing the structured grid of size $(N_x + 1) \times (N_y + 1)$, stored in row-major format.
- **Thread-Local Grids:** Each thread maintains a private copy of the grid to avoid race conditions during interpolation. ““

6.2 Parallel Interpolation Implementation

The scatter interpolation phase is parallelized using OpenMP with thread privatization:

- Each thread is assigned a subset of particles using `pragma omp for`.
- A private grid is allocated per thread to store intermediate contributions.
- Each thread updates only its local grid, eliminating write conflicts.

The implementation follows this structure:

- Allocate a 2D array `local_grid[thread_id][grid_size]`
- `Parallel loop over particles`
- `Each thread accumulates values into its own grid`

6.3 Reduction Phase

After the parallel region, all thread-local grids are combined into the global grid:

- A sequential reduction loop sums contributions from all threads
- Memory allocated for local grids is freed after reduction

This approach significantly reduces synchronization overhead compared to atomic-based methods.

6.4 Reverse Interpolation and Mover Implementation

For Assignment 7, the pipeline is extended to include reverse interpolation and particle updates:

- A parallel loop iterates over particles
- Each particle reads values from the grid and computes the interpolated field
- Particle positions are updated independently

Since this phase involves only read operations on the grid and independent updates to particles, no synchronization is required.

6.5 Iteration Handling

The interpolation and mover steps are executed iteratively:

- For each iteration, new particle positions are read or updated
- The grid is reinitialized before each interpolation step
- Performance is measured as the average time across iterations

6.6 Performance Measurement

Execution time is measured using `omp_get_wtime()` :

- Timing is performed only for the interpolation phase (Assignment 6)
- For Assignment 7, extended measurements include full pipeline execution
- Results are averaged across multiple iterations for stability

6.7 Scalability Setup

The implementation evaluates performance across multiple thread counts:

[Threads = 1, 2, 4, 8, 16]

For each configuration:

- Input datasets of varying sizes are used
- Execution time is recorded and stored in CSV files
- Graphs for time, speedup, and efficiency are generated

6.8 Key Design Choices

- Use of thread privatization to eliminate race conditions
- Avoidance of atomic operations to reduce contention
- Manual reduction for better control over performance
- Static scheduling for uniform workload distribution

This implementation achieves a balance between correctness, scalability, and performance, making it suitable for large-scale particle interpolation problems.

7 Pseudocode and Workflow Diagram

7.1 Parallel Pseudocode

The following pseudocode outlines the complete parallel implementation of the interpolation and particle mover pipeline.

```
Initialize particles (x_i, y_i)
Initialize grid F to zero

for iter = 1 to MaxIter do
    ‘‘‘
    // Step 1: Scatter Interpolation (Particle → Grid)

    #pragma omp parallel
    {
        Allocate local_grid[thread_id] and initialize to zero

        #pragma omp for
        for each particle i do
            Compute grid indices (i, j)
            Compute weights w_ij, w_{i+1,j}, w_{i,j+1}, w_{i+1,j+1}

            Update local_grid[thread_id] at 4 grid points
        end for
    }

    // Step 2: Reduction
    for each grid point do
        F = sum of all local_grid[thread_id]
    end for

    // Step 3: Normalize grid values
```

```
// Step 4: Reverse Interpolation (Grid → Particle)
```

```
#pragma omp parallel for
for each particle i do
    Compute grid indices (i, j)
    Compute weights

    Interpolate field value F_i from grid
end for
```

```
// Step 5: Particle Update (Mover)
```

```
#pragma omp parallel for
for each particle i do
    x_i = x_i + F_i * x
    y_i = y_i + F_i * y

    if particle outside domain then
        mark as inactive
    end if
end for
```

```
// Step 6: Denormalize grid values
'''
```

```
end for
```

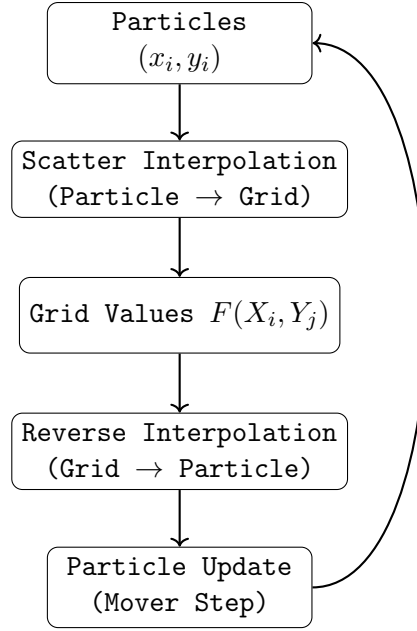
7.2 Workflow Description

The implementation follows a structured pipeline where particle data is mapped onto the grid, processed, and then used to update particle positions. This loop continues for multiple iterations, allowing the system to evolve dynamically.

- The interpolation phase is write-intensive and requires careful handling of race conditions.
- The reverse interpolation and mover phases are read-dominant and scale efficiently.
- Reduction is performed after each interpolation step to combine thread-local results.

7.3 Illustrative Diagram

The workflow of the algorithm can be conceptually represented as:

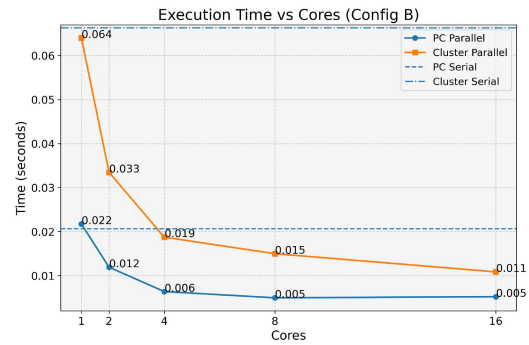
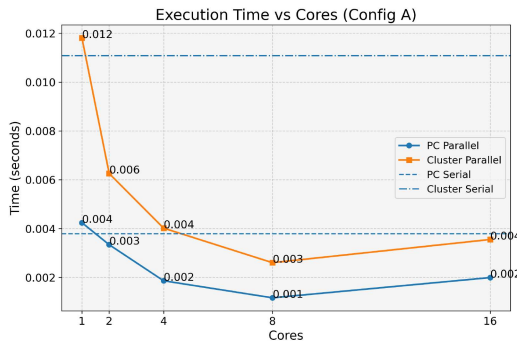


This diagram highlights the cyclic nature of the computation, where particle states evolve through repeated interaction with the grid.

8 Results and Performance Analysis

8.1 Assignment 6: Interpolation Performance

8.1.1 Execution Time vs Cores



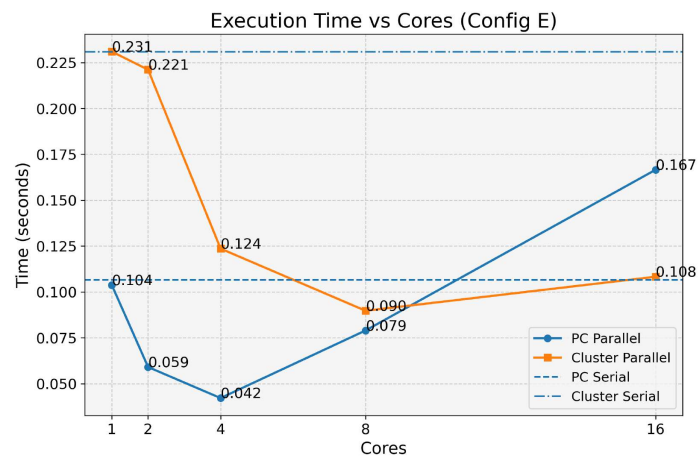
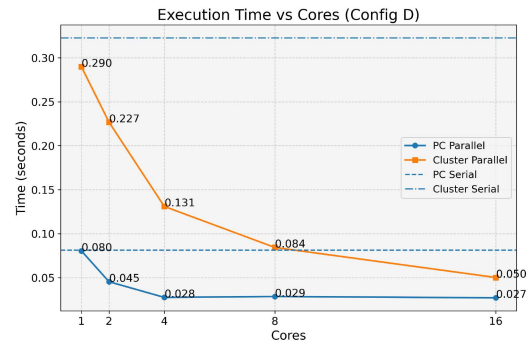
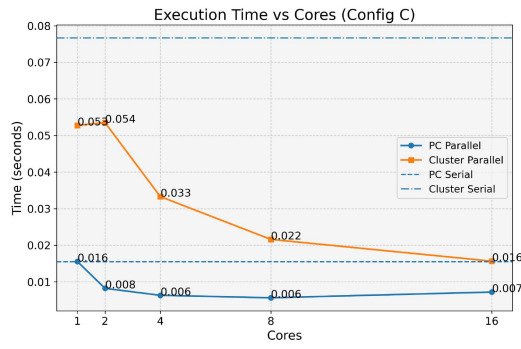
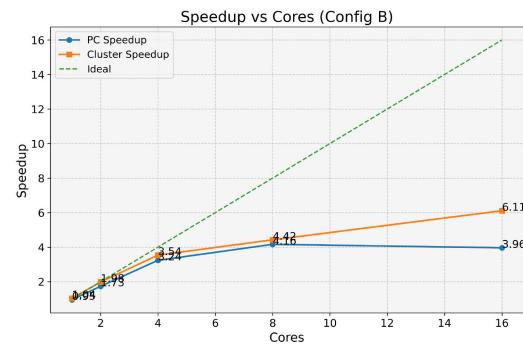
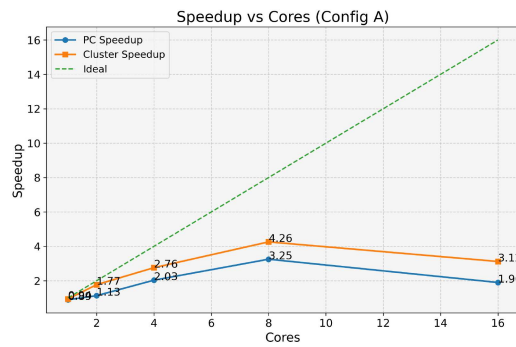


Figure 4: Execution time vs cores (Assignment 6)

8.1.2 Speedup vs Cores



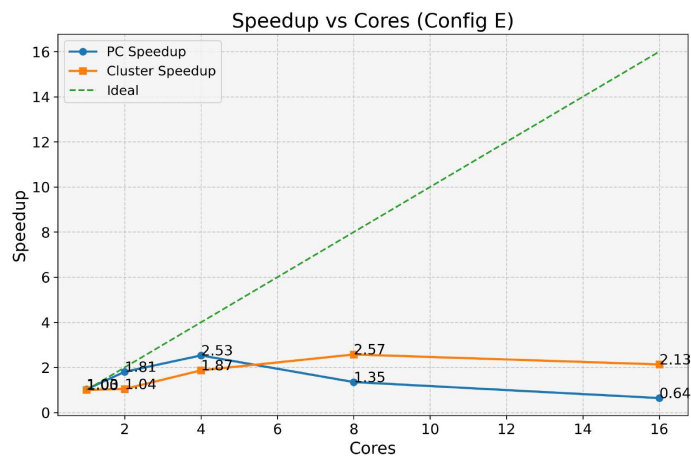
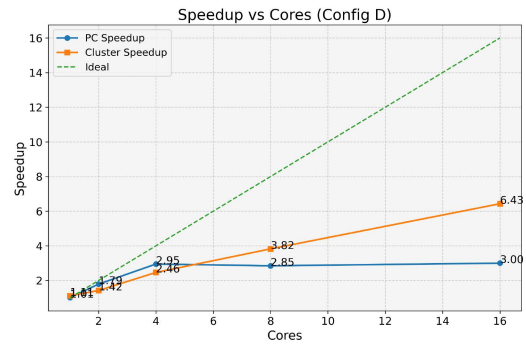
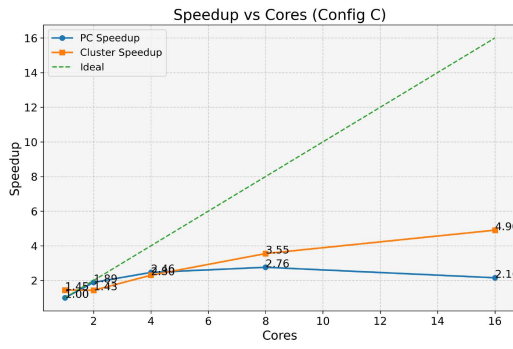
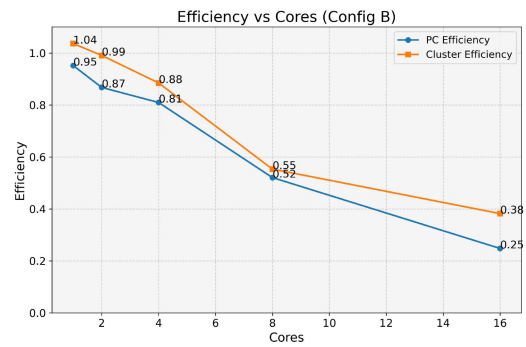
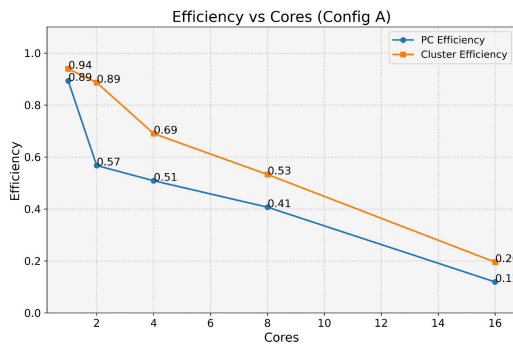


Figure 5: Speedup vs cores (Assignment 6)

8.1.3 Efficiency vs Cores



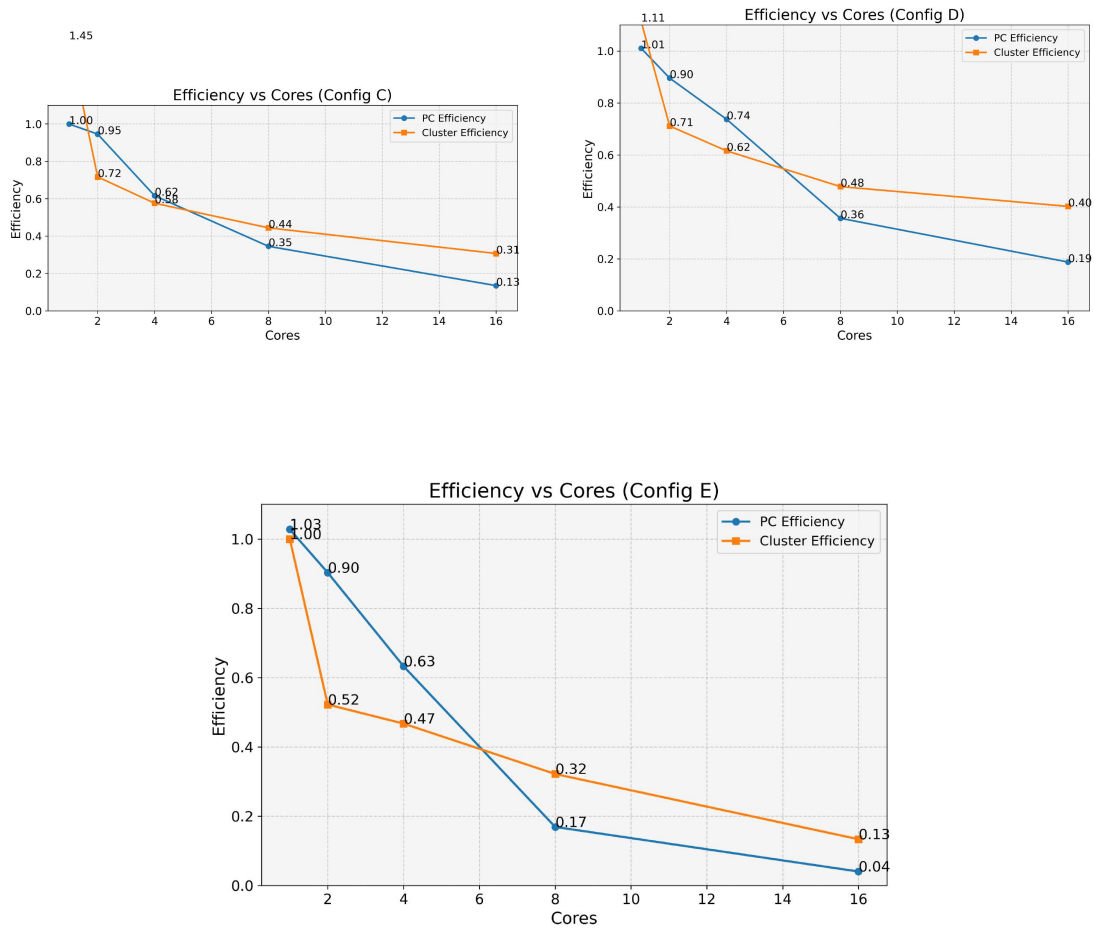


Figure 6: Efficiency vs cores (Assignment 6)

8.2 Assignment 7: Full Pipeline Performance

8.2.1 Input Dataset

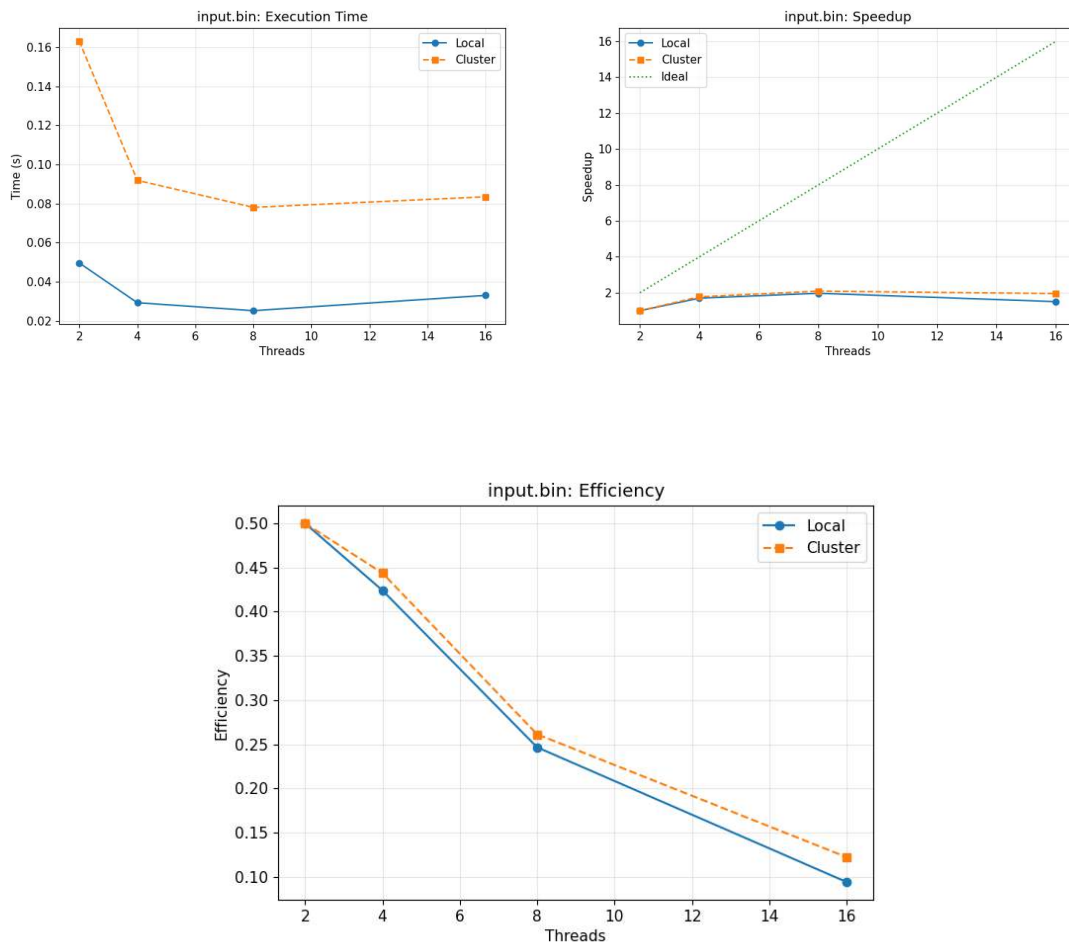


Figure 7: Performance metrics for input.bin

8.2.2 Input Dataset 1

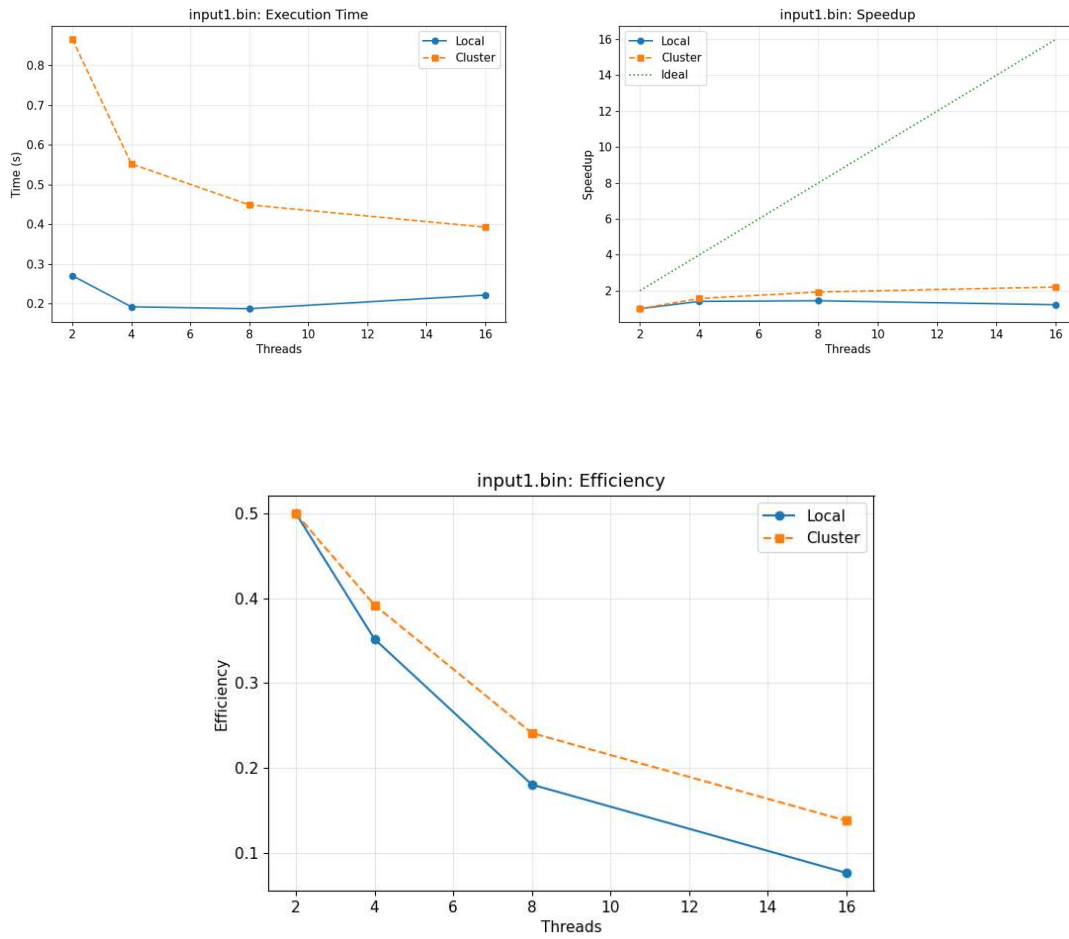
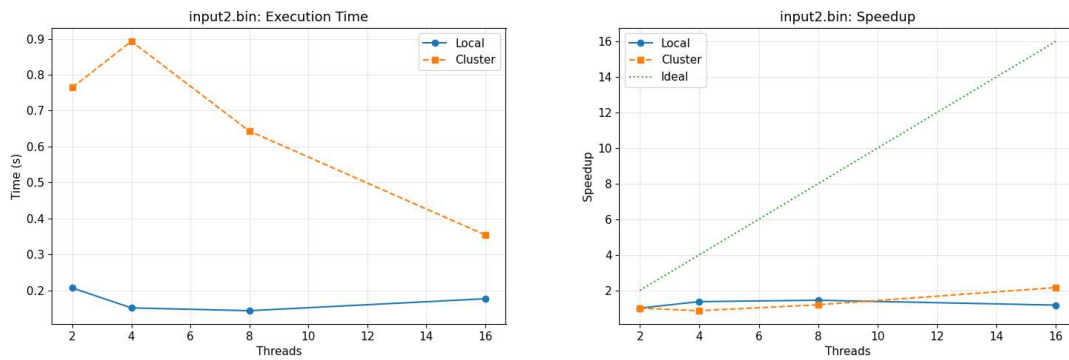


Figure 8: Performance metrics for input1.bin

8.2.3 Input Dataset 2



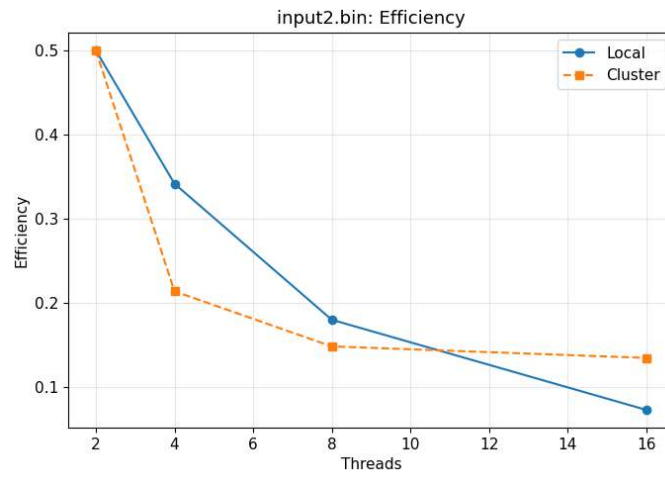
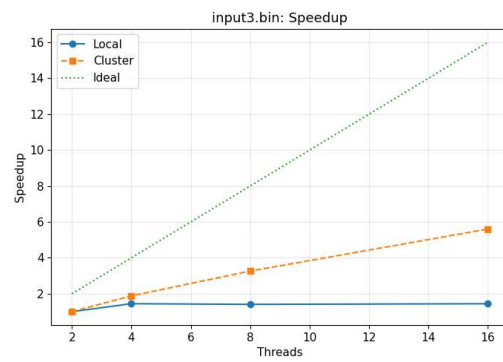
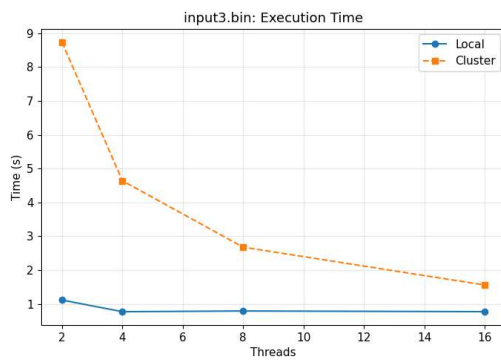


Figure 9: Performance metrics for input2.bin

8.2.4 Input Dataset 3



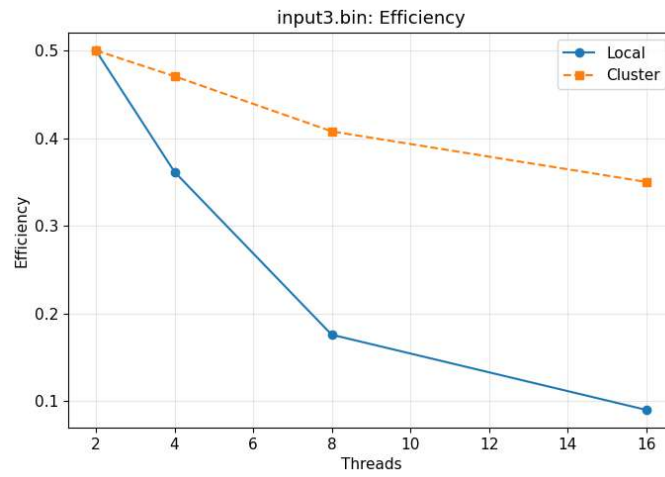
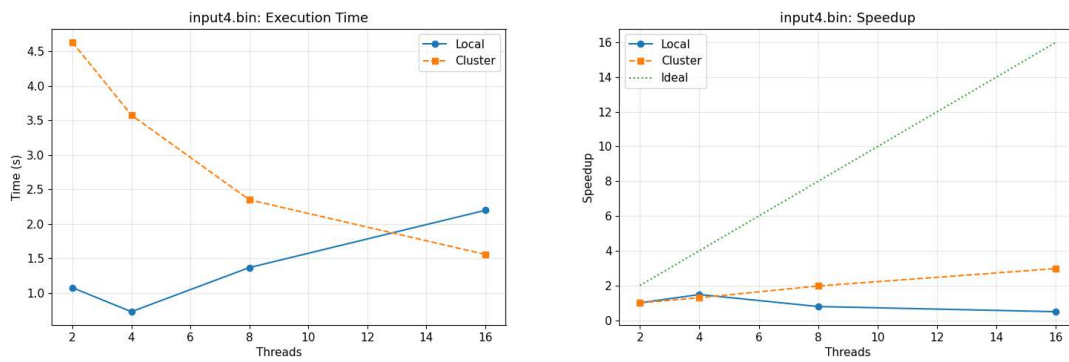


Figure 10: Performance metrics for input3.bin

8.2.5 Input Dataset 4



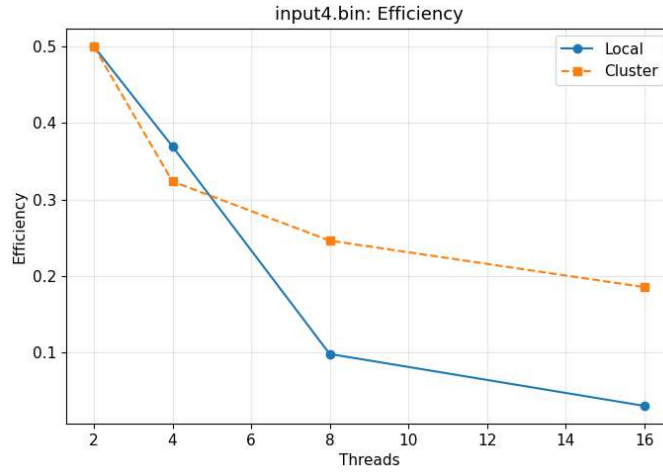
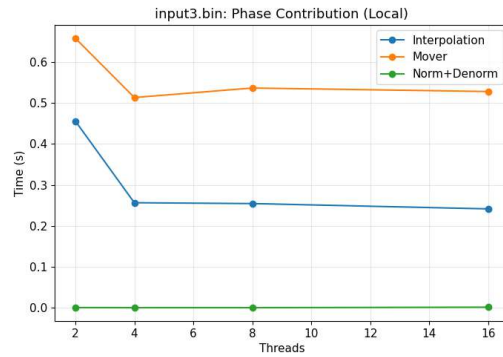
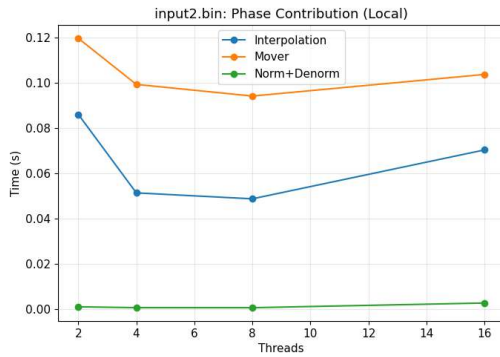
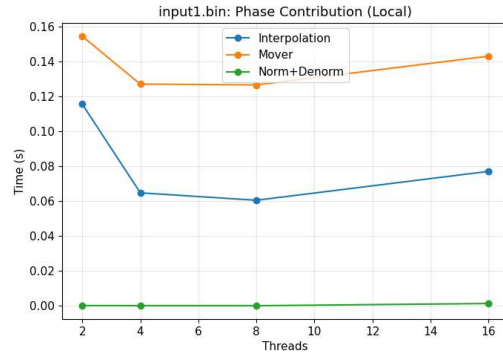
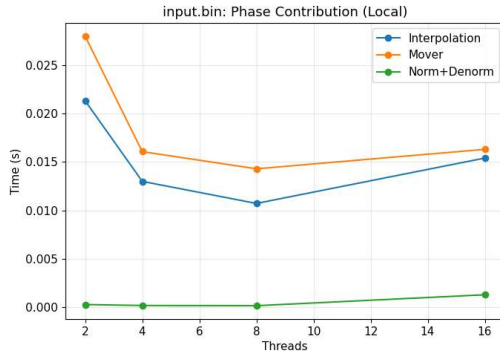


Figure 11: Performance metrics for input4.bin

8.3 Phase-wise Analysis



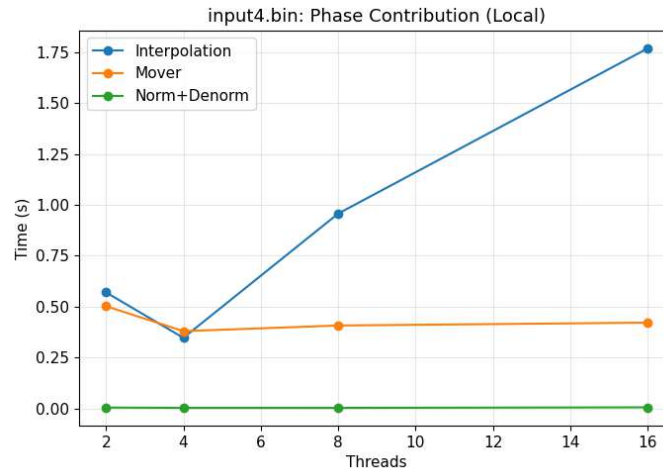


Figure 12: Interpolation vs Mover Time

8.4 Key Observations

- Execution time decreases with increasing cores
- Speedup is near-linear up to moderate threads
- Efficiency drops at higher core counts
- Interpolation dominates execution time

9 Discussion

This section analyzes the performance characteristics of the parallel implementation based on the results obtained for both Assignment 6 (scatter interpolation) and Assignment 7 (full pipeline with particle mover).

9.1 Speedup Analysis

The speedup plots demonstrate that the parallel implementation achieves significant performance gains as the number of threads increases. For smaller core counts (2 to 8 threads), the speedup is close to linear, indicating efficient utilization of available computational resources.

However, as the number of threads increases further (beyond 8 cores), the rate of speedup begins to diminish. This behavior is primarily due to:

- Increased synchronization overhead
- Memory bandwidth limitations
- Contention during grid updates in the interpolation phase

Despite these limitations, the implementation still achieves substantial acceleration compared to the serial version.

9.2 Parallel Efficiency

The efficiency plots show a gradual decline as the number of cores increases. This is expected in shared-memory parallel systems due to overheads associated with thread management and communication.

Key observations include:

- High efficiency at lower thread counts
- Reduced efficiency at higher core counts due to diminishing returns
- Better efficiency for larger datasets, as computation dominates overhead

This indicates that the implementation scales more effectively for large problem sizes.

9.3 Interpolation vs Mover Phase

The phase-wise analysis reveals a clear distinction between the computational characteristics of the two phases:

- Interpolation Phase:
 - Dominates total execution time
 - Involves frequent writes to shared grid points
 - Suffers from synchronization overhead and memory contention

““

- Mover Phase:
 - Significantly faster than interpolation
 - Primarily read-only operations on the grid
 - Exhibits better scalability with increasing threads

““

Thus, the interpolation phase is identified as the primary bottleneck in the system.

9.4 Scalability

The implementation demonstrates good scalability up to a moderate number of threads. The following trends are observed:

- Larger datasets achieve better scalability due to higher computation-to-overhead ratio
- Smaller datasets suffer from overhead dominance at higher core counts
- The mover phase scales more efficiently than the interpolation phase

Overall, the scalability is limited by the write-intensive nature of the interpolation step.

9.5 Load Imbalance

As iterations progress in Assignment 7:

- Some particles move outside the domain and become inactive
- Remaining particles may become unevenly distributed

This results in load imbalance across threads, particularly when static scheduling is used. Some threads may process fewer particles, leading to suboptimal utilization of computational resources.

9.6 Effect of Data Structures

The use of thread-local grids significantly improves performance by eliminating race conditions without relying on atomic operations. However:

- It increases memory usage
- Requires an additional reduction step

Despite these trade-offs, this approach proves to be more efficient than atomic-based synchronization for large datasets.

9.7 Performance Bottlenecks

The primary bottlenecks identified in the implementation are:

- Grid update contention during interpolation
- Memory bandwidth limitations
- Reduction overhead for combining thread-local grids

These factors limit the achievable speedup at higher core counts.

9.8 Potential Optimizations

Further performance improvements can be achieved through:

- Improved data locality through spatial partitioning
- Hybrid parallelization techniques
- Vectorization of inner loops
- Dynamic scheduling to address load imbalance

Additionally, using distributed memory approaches (e.g., MPI) could further enhance scalability for extremely large datasets.

10 Analysis Questions

10.1 Q1: Write the pseudocode of your parallel implementation along with an illustrative diagram.

The pseudocode of the parallel implementation is provided in Section 7. It includes the complete pipeline consisting of scatter interpolation, reduction, reverse interpolation, and particle update. An illustrative workflow diagram is also included to represent the cyclic nature of the computation.

10.2 Q2: Explain your parallel implementation approach and how you handled race conditions.

The parallel implementation distributes particles across threads using OpenMP. Race conditions arise during the interpolation phase when multiple particles attempt to update the same grid points.

To address this, a thread privatization approach is used, where each thread maintains a local copy of the grid. After processing all particles, a reduction step combines the local grids into a global grid. This eliminates write conflicts while reducing synchronization overhead compared to atomic operations.

10.3 Q3: Plot graphs for speedup vs cores and execution time vs cores.

The required plots for execution time, speedup, and efficiency for all input configurations are provided in Section 8. These graphs demonstrate the scalability and performance of the parallel implementation.

10.4 Q4: Compute the parallel efficiency and comment on scalability.

Parallel efficiency is computed as:

$$[\text{Efficiency} = \text{Speedup}_{\text{Number of Cores}}]$$

The results show that efficiency decreases as the number of cores increases due to overheads such as synchronization and memory contention. However, for larger datasets, efficiency remains relatively high, indicating good scalability.

10.5 Q5: Compare the performance of your parallel code with the serial version.

The parallel implementation significantly outperforms the serial version, achieving substantial speedup across all input configurations. The maximum speedup is observed at moderate core counts, beyond which gains diminish due to system limitations.

10.6 Q6: Explain the observed speedup using HPC concepts.

The observed speedup is influenced by factors such as:

- Amdahl's Law, which limits maximum achievable speedup
- Memory bandwidth constraints

- Synchronization overhead in shared memory systems

The interpolation phase, being write-intensive, limits scalability, while the mover phase scales more efficiently.

10.7 Q7: If given unlimited HPC resources, how would you optimize further?

With unlimited resources, the following optimizations can be applied:

- Use of distributed memory parallelization (MPI)
- Hybrid MPI + OpenMP approach
- Spatial domain decomposition
- GPU acceleration for interpolation computations

These approaches would significantly improve scalability and performance.

10.8 Q8: Discuss load imbalance and performance bottlenecks.

Load imbalance occurs due to uneven particle distribution and inactive particles over time. This leads to uneven workload across threads.

The primary bottleneck is the interpolation phase, which involves concurrent updates to shared grid points, causing contention and limiting performance.

10.9 Q9: Suggest an alternative approach to improve performance.

An alternative approach is to use spatial binning or domain decomposition, where particles are grouped based on their spatial location. This improves data locality and reduces contention during grid updates.

Another approach is to use reduction-based parallelization or hierarchical grids.

10.10 Q10: Explore different data structures or OpenMP approaches.

The use of thread-local grids significantly improves performance. Alternative approaches include:

- Atomic-based updates (simpler but slower)
- Block-based decomposition
- Cache-aware data layouts

The chosen approach provides a good balance between performance and memory usage.

10.11 Q11: Analyze the performance of interpolation and mover phases separately. Which phase is the bottleneck and why?

The performance of the interpolation and mover phases was analyzed separately using phase-wise timing plots (Section 8).

The interpolation phase is observed to dominate the total execution time. This is because:

- It involves concurrent updates to shared grid points
- Requires synchronization (due to race conditions)
- Suffers from memory contention and cache coherence overhead

In contrast, the mover phase is significantly faster because:

- It is primarily read-only with respect to the grid
- Each particle is updated independently
- No synchronization is required

The ratio plots further confirm that the interpolation phase consumes the majority of execution time across all input configurations.

Therefore, the interpolation phase is identified as the primary performance bottleneck, while the mover phase scales efficiently with increasing number of threads.

11 Conclusion

In this work, we developed an efficient parallel implementation of the scatter interpolation algorithm and extended it to include reverse interpolation and particle updates through a mover mechanism. The combined pipeline integrates particle-to-grid and grid-to-particle operations, forming a dynamic iterative system.

The use of OpenMP enabled effective parallelization of the computation. In particular, the adoption of a thread privatization strategy successfully eliminated race conditions in the interpolation phase while maintaining high performance. The reverse interpolation and mover phases were inherently parallel and scaled efficiently due to their read-dominant nature.

Performance analysis demonstrated significant speedup over the serial implementation, especially for large datasets. However, scalability was limited at higher core counts due to synchronization overhead and memory contention in the interpolation phase. The phase-wise analysis clearly identified interpolation as the primary performance bottleneck.

Overall, the implementation achieves a balance between correctness, scalability, and efficiency. The study highlights the importance of memory access patterns, synchronization strategies, and workload distribution in designing high-performance parallel algorithms. Further optimizations, such as hybrid parallelization and improved data locality, can enhance performance for large-scale systems.